



Attention Is All You Need

김재희, 신아영

목차

- 0. Abstract
- 1. Introduction
- 2. Background
- 3. Model Architecture
 - 3.1 Encoder and Decoder Stacks
 - 3.2 Attention
 - 3.2.1 Scaled Dot-Product Attention
 - 3.2.2 Multi-Head Attention
 - 3.2.3 Applications of Attention in our Model
 - 3.3 Position-wise Feed-Forward Networks
 - 3.4 Embeddings and Softmax
 - 3.5 Positional Encoding
- 4. Why Self-Attention
- 5. Training
 - 5.1 Training Data and Batching
 - 5.2 Hardware and Schedule
 - 5.3 Optimizer
 - 5.4 Regularization
- 6. Results
 - 6.1 Machine Translation
 - 6.2 Model Variations
 - 6.3 English Constituency Parsing
- 7. Conclusion



0. Abstract



0. Abstract

- 기존 주류 시퀀스 변환 모델들: RNN or CNN 기반
- 본 연구에서는 RNN, CNN 배제하고 어텐션 매커니즘만을 기반으로 한 트랜스포머 제안
- 두 가지 기계 번역 실험 결과, 해당 모델이 병렬화하기 더 쉽고 훈련 시간이 적게 들며 품질 면에서도 우수했음
- 번역뿐만 아니라 영어 구문 분석 과제에도 성공적으로 적용됨 -> 일반화 가능

1. Introduction



1. Introduction

기존 state of the art 방식

- RNN, LSTM, GRU는 기계 번역 및 언어 모델링 표준이었음

한계

- 순환 모델은 현재 은닉 상태 h_t 를 계산할 때 반드시 h_{t-1} 을 필요로 함
- 이러한 특성 때문에 병렬화 불가능 -> 시퀀스 길이가 길어질수록 메모리 제약으로 인해 배치 처리도 제한됨
- 최근 연구는 인수분해 기법과 조건부 계산을 통해 계산 효율성을 개선했지만 여전히 병렬화 불가능이라는 제약 존재

대안

- 어텐션 메커니즘은 기존에도 다양한 작업에서 뛰어난 시퀀스 모델링 및 변환 모델의 핵심 요소였음
- 하지만 극소수의 경우를 제외하면 어텐션은 대부분 순환 신경망과 함께 사용됨
- 본 연구에서는 순환 신경망을 배제하고 어텐션 메커니즘에만 의존하는 트랜스포머를 제안 -> 높은 수준의 병렬화 가능, 번역 품질에서 새로운 최고 기록 달성

2. Background



2. Background

다른 모델들

- Extended Neural GPU), ByteNet, ConvS2S 같은 병렬 계산을 목표로 한 모델들도 존재
- 이들은 모두 CNN을 기본 구성 요소로 사용함
- 하지만 이 모델들에서는 입출력 신호를 연관시키는 데 필요한 연산 횟수가 위치 간 거리에 따라 증가함 -> 멀리 떨어진 위치 간 의존 관계를 학습하기 어려움

e.g. ConvS2S: 선형적으로 증가, ByteNet: 로그 형태로 증가

트랜스포머

- 트랜스포머의 경우 연산 횟수가 상수 횟수만큼 적음(1회)
- 하지만 여러 위치들을 평균 내는 과정에서 유효 해상도가 낮아짐(정보가 뭉개짐) -> 이는 멀티 헤드 어텐션으로 상쇄

2. Background

셀프 어텐션

- 셀프 어텐션: 하나의 시퀀스 내 서로 다른 위치들을 연관시켜 그 시퀀스의 표현을 계산하는 메커니즘
- 기존에도 다양한 과제에서 성공적으로 활용되어 옴

엔드투엔드 메모리 네트워크

- 순환 어텐션 메커니즘을 기반으로 함
- 간단한 언어 질의응답 및 언어 모델링 과제에서 좋은 성능을 보임

트랜스포머

- RNN이나 CNN을 전혀 사용하지 않고 오직 셀프 어텐션만으로 입력과 출력의 표현을 계산하는 최초의 모델

3. Model Architecture



3. Model Architecture

- 인코더-디코더 구조
 - 인코더: 입력 시퀀스를 컴퓨터가 이해할 수 있는 표현의 시퀀스 z 로 변환함
 - 디코더: z 가 주어지면 한 번에 한 원소씩 출력 시퀀스를 생성함
 - 각 단계에서 모델은 자기회귀적이며 다음 심볼을 생성할 때 이전에 생성된 심볼들을 추가 입력으로 사용함
- e.g. I love you -> 나는 너를 사랑해: “사랑해” 를 생성할 때 “나는 너를” 참조
- 인코더와 디코더 모두 셀프 어텐션과 포인트와이즈 완전연결층을 쌓아 올린 구조를 따름

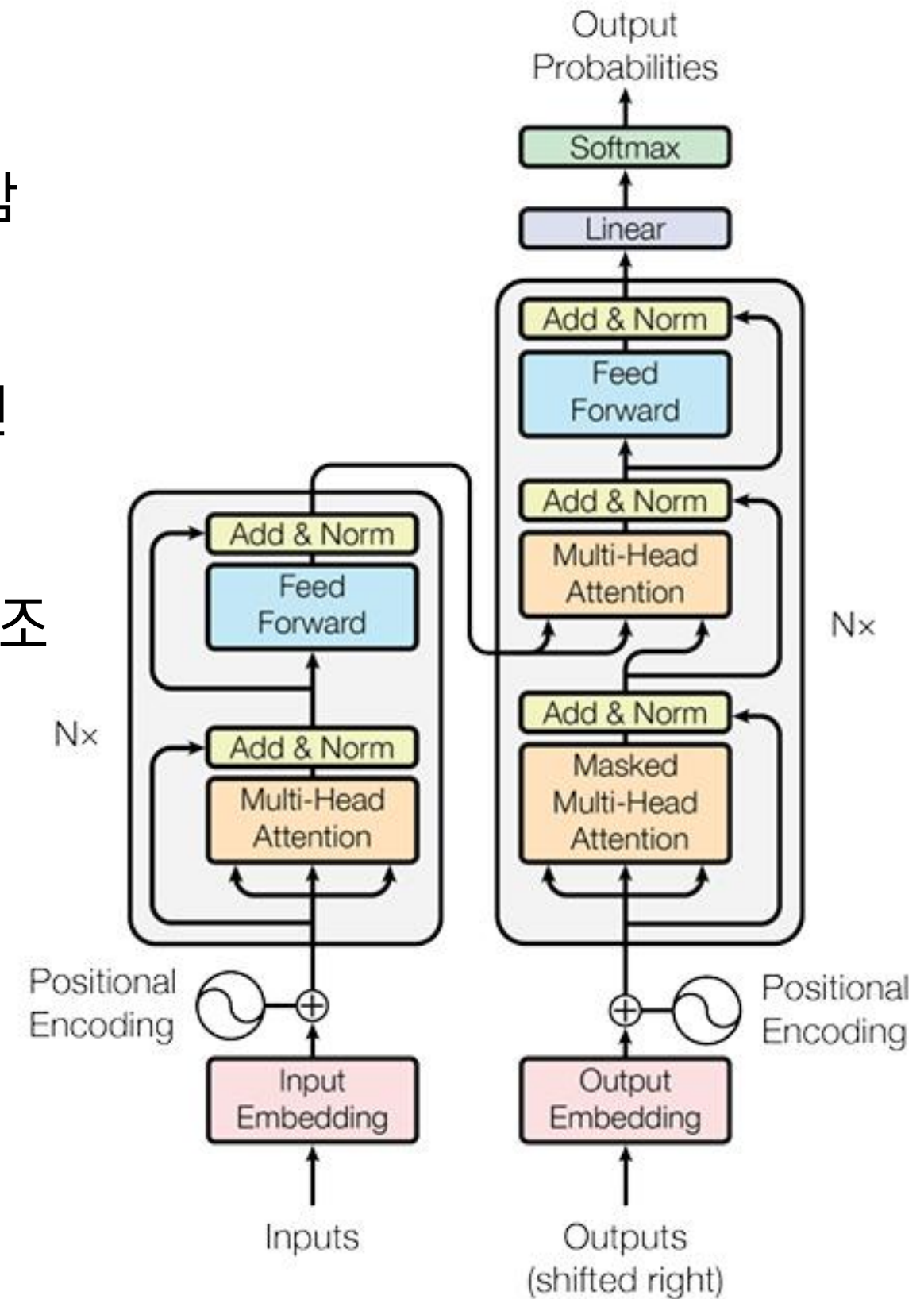


Figure 1: The Transformer - model architecture.

3.1 Encoder and Decoder Stacks

인코더

- (N = 6) 6층 반복 구조
- 각 층은 두 개의 서브층(멀티 헤드 셀프 어텐션, FFN)으로 이루어짐
- 두 서브층 각각에 잔차 연결과 층 정규화 수행
- 각 서브층 출력은 $\text{LayerNorm}(x + \text{Sublayer}(x))$
- 잔차 연결을 하기 위해서 모델 내 모든 서브층과 임베딩 층은 차원 $d_{model} = 512$ 크기로 통일 (입력과 출력 크기가 같아야 하기 때문)

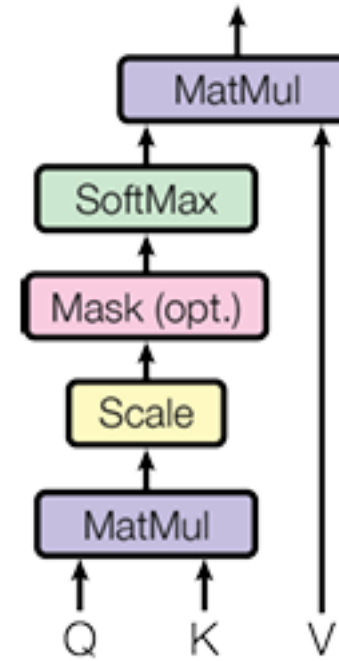
디코더

- (N = 6) 6층 반복 구조
- 각 층은 세 개의 서브층(마스크드 멀티 헤드 셀프 어텐션, 인코더-디코더 어텐션, FFN)으로 이루어짐
- 세 서브층 각각에 잔차 연결과 층 정규화 수행
- 마스크드 멀티 헤드 셀프 어텐션: 학습 시 병렬로 데이터를 처리할 때, 뒤에 나올 미래 단어들을 미리 못보도록 마스킹함
e.g. “나는 너를 사랑해” 생성할 때: “너를” 생성할 시점에는 “나는”만 볼 수 있음

3.2 Attention

- 하나의 쿼리와 키-값 쌍을 매핑하는 것
- 쿼리, 키, 값 출력은 모두 벡터
- 출력은 값들의 가중합으로 계산되는데, 각 값에 할당되는 가중치는 해당 키와 쿼리 간의 호환성 함수를 통해 계산됨 (쿼리와 키가 얼마나 잘 맞는지)

Scaled Dot-Product Attention



Multi-Head Attention

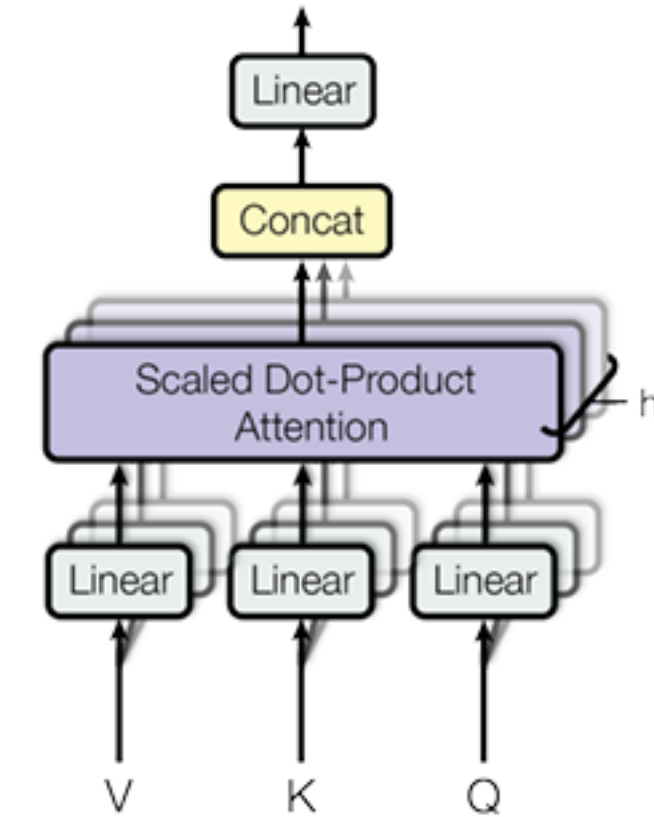


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

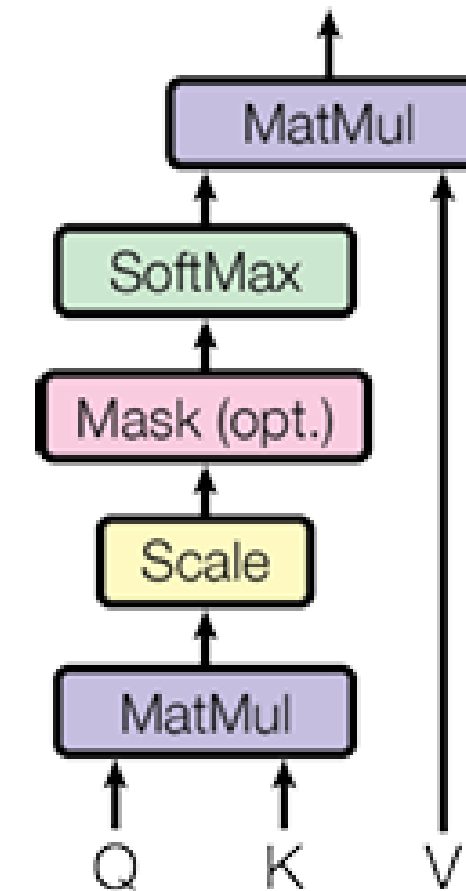
3.2.1 Scaled Dot-Product Attention

- 입력: 차원 d_k 의 쿼리와 키, 차원 d_v 의 값으로 구성

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- 쿼리와 모든 키의 내적(dot product)을 계산한 뒤, 각각을 $\sqrt{d_k}$ 로 나누고,
소프트맥스 함수를 적용해 값들에 대한 가중치를 얻음
- 여러 쿼리에 대해 동시에 어텐션 계산을 함 -> 이 쿼리들을 하나로 묶어 행렬 Q로 만듦
- 키와 값 역시 각각 행렬 K와 V로 묶음

Scaled Dot-Product Attention

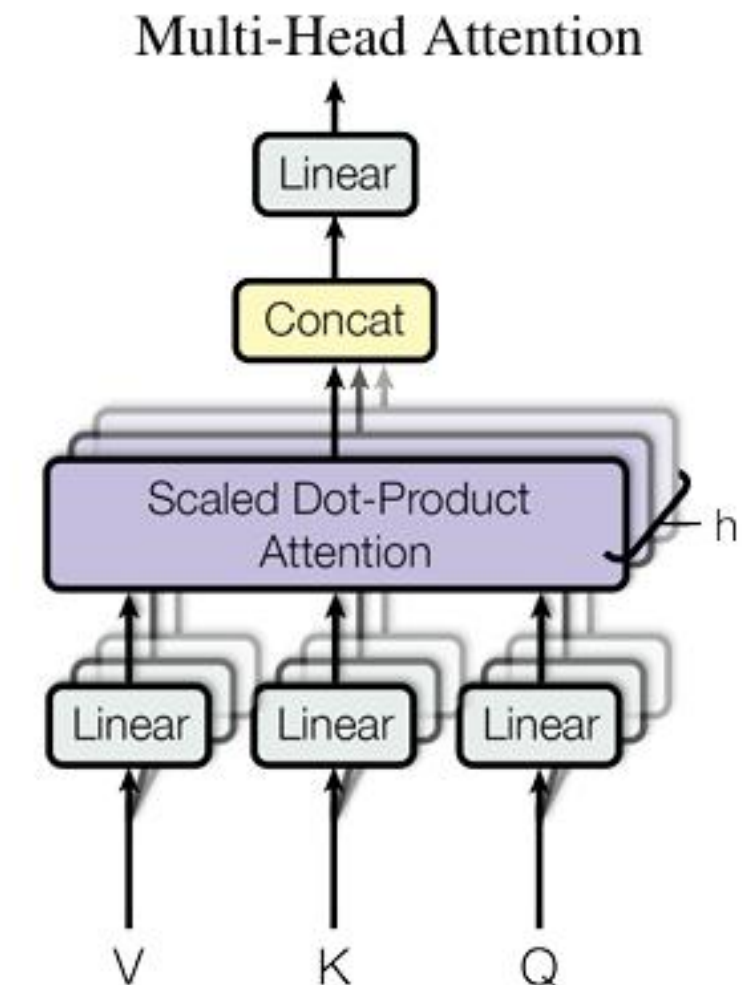


3.2.1 Scaled Dot-Product Attention

- 가장 흔히 사용되는 어텐션 함수는 가산 어텐션과 Dot-Product(이 논문에서 채택됨)
- 두 방식은 이론적 복잡도가 비슷하지만 Dot-Product 같은 경우 최적화된 행렬 곱셈 코드를 활용할 수 있기에 훨씬 더 빠르고 공간 효율적임
- d_k 값이 작을 때는 두 메커니즘 성능이 비슷하지만 d_k 값이 커질수록 Dot-Product보다 가산 어텐션이 더 좋은 성능을 보임
- 입력값이 너무 크면 하나의 값에 치중되어서 $\sqrt{d_k}$ 로 나누는 것
e.g. [10, 20, 30] -> [0, 0.000045, 0.999955] -> 그래디언트가 거의 0에 가까워져서 학습이 잘 안 됨

3.2.2 Multi-Head Attention

- 단일 어텐션 함수를 수행하는 대신 쿼리, 키, 값을 서로 다른 선형 사상(projection)으로 각각 h 번 선형 사영하는 것이 더 유리함
- 이렇게 사영된 쿼리, 키 값에 대해 어텐션 함수를 병렬로 수행, 이것들을 연결한 뒤 다시 한 번 사영하여 최종 값을 얻음
- 멀티 헤드 어텐션은 모델이 서로 다른 위치에서 서로 다른 표현 부분공간의 정보에 동시에 주의를 기울일 수 있도록 해줌
e.g. 문법적 관계(주어-동사), 의미적 유사성(유의어/반의어), 지시 관계 등
- 본 연구에서는 $h = 8$ 개의 헤드를 사용함
- 각 헤드의 차원이 축소되기 때문에(기존 512차원 $\rightarrow 512/8 = 64$ 차원) 전체 계산 비용은 전체 차원을 사용하는 단일 헤드 어텐션과 비슷함



3.2.3 Applications of Attention in our Model

- 멀티 헤드 어텐션은 세 가지 방식으로 사용됨

종류	쿼리 출처	키/값 출처	역할
인코더-디코더 어텐션	이전 디코더 층	인코더 출력	디코더가 원문 참고
인코더 셀프 어텐션	이전 인코더 층	이전 인코더 층	입력 문장 내에서 단어들 간 관계 파악
마스크드 디코더 셀프 어텐션	이전 디코더 층	이전 디코더 층(미래 디코더 층은 마스킹)	지금까지 생성된 단어들 간 관계 파악

- 마스킹 할 때 미래 값들은 $-\infty$ 로 설정함: $\text{softmax}(-\infty)=0 \rightarrow$ 가중치가 0이 되어 해당 위치는 무시됨

3.3 Position-wise Feed-Forward Networks

- FFN층에는 파라미터 상당수가 포함됨. 주로 feature를 변형하고 새로운 표현을 만드는 일을 함
- 각 위치에 대해 개별적으로, 동일하게 적용됨

e.g. self attention이 “I love you”라는 문장 전체에 한 번 적용된다면 FFN은 “I”, “love”, “you” 각각에 적용됨

- 다만 층마다 서로 다른 파라미터 사용

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- 각 단계에 의해 512차원 -> 2048차원으로 확장, ReLU 적용, 2048차원 -> 512차원으로 축소 -> 각 단어 벡터를 한 번 더 비선형적으로 가공하는 것 -> 표현력이 늘어남

3.4 Embeddings and Softmax

- 본 연구에서는 임베딩을 통해 입력 토큰과 출력 토큰을 d_{model} 차원 벡터로 변환함
- 소프트맥스 함수를 사용해 디코더 출력을 다음 토큰에 대한 예측 확률로 변환함
- 트랜스포머에서는 임베딩 층과 소프트맥스 이전 선형 변환 사이에 동일한 가중치 행렬을 공유함 -> 파라미터 수 줄이고 학습을 더 효율적으로 만듦

3.5 Positional Encoding

문제 상황

- 어텐션엔 위치 정보가 따로 없음
 - e.g. “나는 너를 사랑해 ” 와 “사랑해 너를 나는“ 두 문장에서 단어들 간 관계 점수는 완전히 똑같음
- RNN 같은 경우엔 순서대로 계산을 하므로 모델이 순서를 자연스럽게 배움

해결책

- 모델이 시퀀스 순서를 활용할 수 있게 하도록 입력 임베딩에 위치 인코딩을 더해주는 방법 사용
- 위치 인코딩은 $\sin$, $\cos$ 함수로 만들어짐

3.5 Positional Encoding

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

- pos: 단어의 위치
- i: 차원
- 짝수 차원엔 sin, 홀수 차원엔 cos 사용
- Sin, cos을 사용하면 상대적 위치를 선형적으로 표현 가능 -> 상대적 거리 관계 학습하기 쉬워짐
- 본 연구에서는 학습된 위치 인코딩도 시도해 봤지만 sin, cos 사용 방식이 더 긴 문장에 대응 가능함

4. Why Self-Attention



4. Why Self-Attention

1. 층별 총 계산 복잡도
2. 병렬화 가능성
3. 네트워크 내 장거리 경로 길아: 입력과 출력 시퀀스 내 임의의 위치 조합 간 경로가 짧을수록 장거리 의존 관계를 학습하기 더 쉬워짐

4. Why Self-Attention

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

- 시퀀스 길이 n 이 표현 차원 d 보다 작을 때는 셀프 어텐션 층이 Recurrent 층보다 더 빠름
- 하지만 문장이 아주 길어지면(n 이 커지면) 셀프 어텐션에 부담이 될 수 있으므로 셀프 어텐션이 각 출력 위치를 중심으로 한 크기 r 의 주변 영역만 고려하도록 제한할 수 있음, 하지만 경로 길이가 $O(n/r)$ 로 늘어날 수 있음
- 경로 길이 측면에서 CNN은 셀프 어텐션보다 구조적으로 손해
- Separable convolution이라는 보다 개선된 CNN 기법을 써도 계산 복잡도가 “셀프 어텐션+FFN”과 같음
- RNN, CNN은 내부에서 무슨 일이 벌어지는지 알기 어렵지만 어텐션에서는 단어 간 가중치 값을 시각화 가능

05. Training



5.1 Training Data & Batching

WMT 2014 EN-DE

약 4.5M sentence pairs
BPE 사용
약 37K shared source-target vocabulary

WMT 2014 EN-FR

약 36M sentence pairs
32K word-piece vocabulary
EN-DE보다 훨씬 큰 데이터셋

Batching

비슷한 길이의 문장끼리 묶음
약 25K source tokens + 25K target
tokens / batch

핵심: 번역 실험은 대규모 WMT 데이터셋에서 진행했고, 배치는 문장 개수보다 token 수 기준으로 구성했다.

5.2-5.3 Hardware, Schedule & Optimizer

Hardware / Schedule

8 NVIDIA P100 GPUs
base: 100K steps, 약 12h
big: 300K steps, 약 3.5d

Adam Optimizer

$\beta_1 = 0.9$, $\beta_2 = 0.98$
 $\epsilon = 10^{-9}$
warmup_steps = 4000

Learning Rate Schedule

초반 4000 step은 선형 증가
이후 step 수의 inverse square root에
비례해 감소

warm-up

decay

Learning rate는 처음부터 크게 주지 않고, warm-up 후 점진적으로 감소시켜 안정적으로 학습한다.

5.4 Regularization

Residual Dropout

각 sub-layer 출력에 dropout 적용
그 후 sub-layer input과 더하고 normalization
embedding + positional encoding 합에도 적용

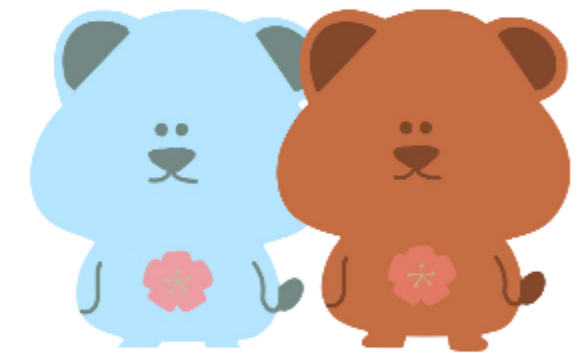
Label Smoothing

$\epsilon_{ls} = 0.1$
모델이 정답 하나에 과도하게 확신하는 것을 완화
perplexity는 나빠지지만 accuracy와 BLEU는 개선

정리

Training section에서 명시된 정규화는 residual dropout과 label smoothing이 핵심이다.

6-1. Results



6.1 Machine Translation Results

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

EN-DE

Transformer big: 28.4 BLEU
이전 best + ensembles보다 2 BLEU 이상 높음

EN-FR

Transformer big: 41.8 BLEU
Table 2 기준 new single-model SOTA

Takeaway

Attention-only architecture가
recurrent/convolutional model보다 높은 번역 성
능을 보였다.

Source: Vaswani et al., "Attention Is All You Need", arXiv:1706.03762v7

6.1 Translation Quality vs. Training Cost

Cost comparison

base: 3.3×10^{18} FLOPs
big: 2.3×10^{19} FLOPs
경쟁 모델 대비 낮은 training cost

Inference setting

base: last 5 checkpoints 평균
big: last 20 checkpoints 평균
beam size = 4, length penalty $\alpha = 0.6$

Cost estimation

training time $\times$ GPU 수 $\times$ GPU single-precision capacity로 FLOPs 추정

성능만 높은 것이 아니라, 병렬화 가능한 구조 덕분에 학습 비용도 줄였다는 점이 결과의 핵심이다.

6-2. Model Variations



6.2 Model Variations: What Changed?

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$	
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65	
(A)					1	512	512				5.29	24.9	
					4	128	128				5.00	25.5	
					16	32	32				4.91	25.8	
					32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58	
					32					5.01	25.4	60	
(C)	2									6.11	23.7	36	
	4									5.19	25.3	50	
	8									4.88	25.5	80	
	256				32	32				5.75	24.5	28	
					128	128				4.66	26.0	168	
	1024								5.12	25.4	53		
			1024						4.75	26.2	90		
			4096										
(D)							0.0				5.77	24.6	
							0.2				4.95	25.5	
							0.0			4.67	25.3		
							0.2			5.47	25.7		
(E)	positional embedding instead of sinusoids									4.92	25.7		
big	6	1024	4096	16				0.3	300K	4.33	26.4	213	

A. Heads

single-head는 best setting보다 0.9 BLEU 낮음
heads가 너무 많아도 quality drop

B. d_k

attention key size를 줄이면 model quality 하락

C/D. Scale & Dropout

bigger models are better
dropout is very helpful in avoiding over-fitting

E. Positional Encoding

learned positional embeddings $\approx$ sinusoidal
positional encoding

Source: Vaswani et al., "Attention Is All You Need", arXiv:1706.03762v7

6.2 Attention Heads & d_k

Heads 실험

1 head: 24.9 BLEU
4 heads: 25.5
8 heads: 25.8
16 heads: 25.8
32 heads: 25.4

핵심 결과

single-head는 best setting보다 0.9 BLEU 낮음
heads가 너무 많아져도
quality drop

d_k 실험

attention key size d_k 를 줄이면 model quality 하락

논문 해석

query-key compatibility 계산은 쉽지 않음
더 정교한 compatibility function이 도움될 가능성

정리: multi-head attention은 중요하지만, head 수를 무조건 늘리는 것이 답은 아니며 각 head의 차원도 함께 고려해야 한다.

6.2 Model Size, Dropout & Position

Model size

bigger models are better
base: 65M params, 25.8 BLEU
big: 213M params, 26.4 BLEU

d_model / d_ff

model dimension과 feed-forward dimension을 키우면
전반적으로 BLEU 상승

Dropout

dropout은 over-fitting 방지에
매우 중요
Pdrop을 바꾸면 성능 차이가
나타남

Positional encoding

learned positional embedding
≈ sinusoidal positional
encoding
최종적으로 sinusoidal 선택

정리: Transformer 성능은 attention 구조 하나만이 아니라 충분한 model capacity, 적절한 dropout, 위치 정보 설계가 함께 만든 결과다.

6.2 Ablation: Main Interpretation

Multi-head attention

여러 representation subspace
와 위치 정보를 동시에 볼 수
있음

Key dimension

d_k 가 너무 작으면
compatibility 계산이 어려워 성
능 저하

Model capacity

d_{model} , d_{ff} 가 커질수록 전반
적으로 BLEU 향상

Regularization

dropout은 overfitting 방지에
중요

결론: Transformer의 성능은 attention-only 구조뿐 아니라 head 수, dimension, model size, dropout의 균형에
서 나온다.

6.3 English Constituency Parsing

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Set of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

Task

번역이 아닌 constituency parsing으로 일반화 확인
출력은 구조 제약이 강하고 입력보다 길 수 있음

Setting

4-layer Transformer, d_model=1024
WSJ only: 약 40K sentences
semi-supervised: 약 17M sentences

Result

WSJ only: 91.3 F1
semi-supervised: 92.7 F1
Task-specific tuning이 적어도 좋은 결과

Source: Vaswani et al., "Attention Is All You Need", arXiv:1706.03762v7

6. Results Summary

Translation

EN-DE: 28.4 BLEU
EN-FR: 41.8 BLEU
두 번역 task에서 높은 성능 달성

Efficiency

recurrent/convolutional
architecture보다 병렬화가 쉬움
낮은 training cost로 경쟁력 확보

Ablation

multi-head, d_k , model scale,
dropout이 성능에 영향
head 수는 적절한 균형 필요

Generalization

English constituency parsing에서도 좋은 결과
번역 외 task로 확장 가능성 확인

결론: Transformer의 실험 결과는 “attention-only 구조도 성능과 효율성을 모두 달성할 수 있다”는 논문의 핵심 주장을 뒷받침한다.

07. Conclusion



07. Conclusion & Future Work

Main contribution

Transformer는 recurrence와 convolution 없이 attention만으로 구성된 sequence transduction model이다.

Empirical results

WMT 2014 EN-DE/EN-FR에서 새로운 SOTA를 달성했고, 특히 EN-DE에서는 ensembles까지 넘었다.

Future work

Text 외 image/audio/video로 확장
local/restricted attention으로 큰 입력 처리
generation을 less sequential하게 만들기

=> 이 논문은 “attention을 보조기법으로 쓰는 모델”이 아니라, attention 자체를 중심 구조로 세운 모델을 실험으로 검증했다.

THANK YOU

Q&A / Discussion

